جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

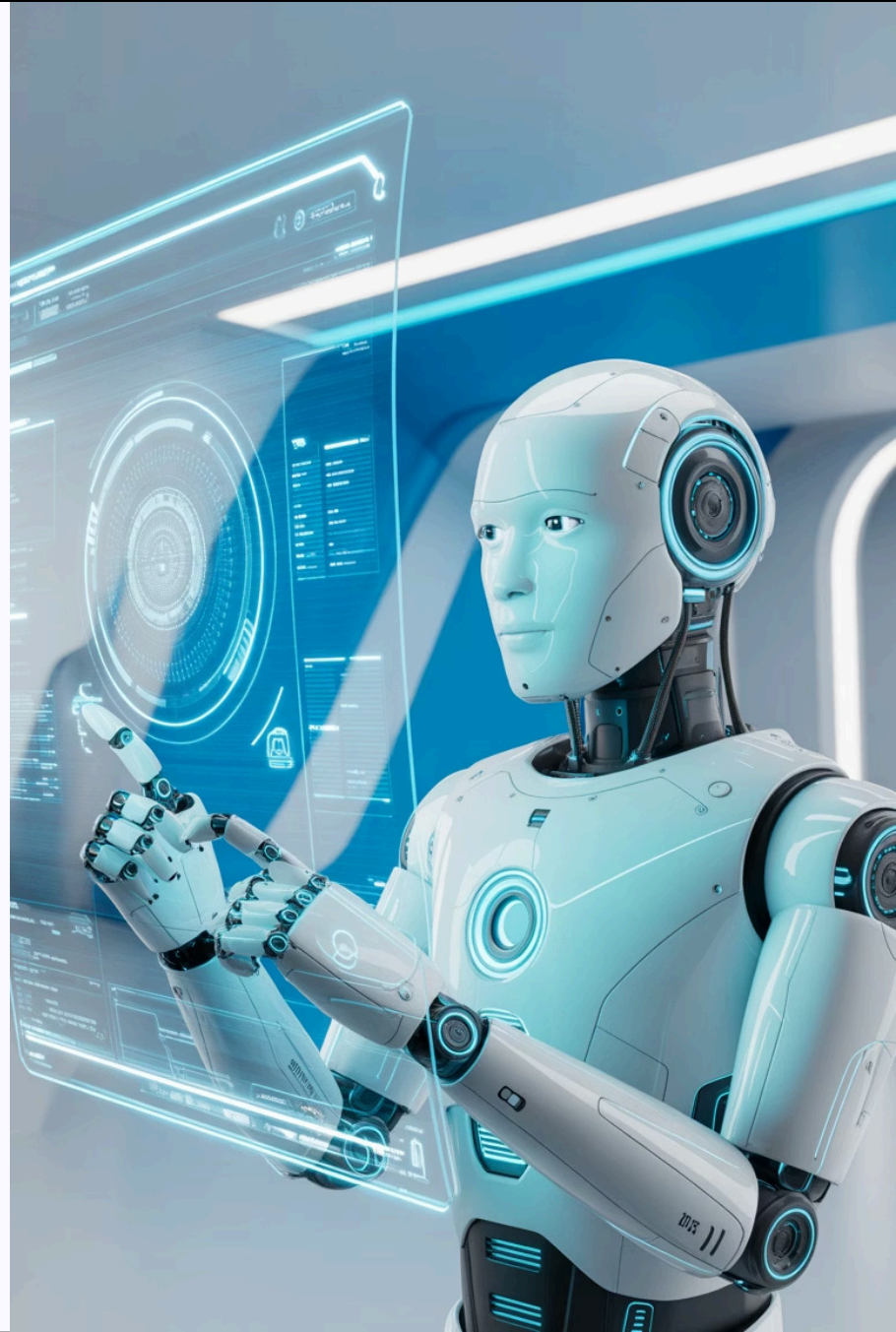
Introduction to Artificial Intelligence (SE 444)

Lecture 9: Automated Planning

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture explores **Classical Planning in AI**, covering **PDDL (Planning Domain Definition Language)**, state representation, action schemas, and planning algorithms. You'll learn how AI agents use factored representations to **solve complex planning problems**, from basic **forward/backward search** to hierarchical planning and handling uncertainty.



What We'll Cover 📚

01

Classical Planning Foundations

Factored representations, PDDL language, and core planning concepts

02

Planning Algorithms

Forward search, backward search, and SAT-based approaches

03

Heuristics & Optimization

Domain-independent heuristics and search improvements

04

Hierarchical Planning

High-level actions and decomposition strategies

05

Nondeterministic Domains

Partial observability, contingency planning, and online replanning

06

Time & Resources

Scheduling problems with temporal and resource constraints

🧩 1️⃣ The Problem: How Do We Describe the World for a Planner?

In **Classical Planning**, an AI agent needs to reason about the world —
✅ what is *true now* and
🎯 what should be *true after acting*.

To do this, we must represent the **state of the world** in a **precise, machine-readable way**.

🔍 Challenge:

- The world changes over time.
- States must capture all *relevant facts* about the current situation.
- We need a representation that is **logical**, **compact**, and **unambiguous**.

💡 Solution → Use PDDL (Planning Domain Definition Language)

→ a formal language based on **First-Order Logic** that describes:

- **Objects** in the world
- **Predicates (fluents)** that describe properties or relations
- **States** as sets of *true facts*

Classical Planning

🌟 **Classical Planning** refers to the **process of finding a sequence of actions** that transforms an **initial state** into a **goal state**, assuming a **simplified environment** characterized by the following properties:

- 🧩 **Discrete:** The world is made up of a finite set of distinct states and actions.
- 🎯 **Deterministic:** Each action has a predictable and unique outcome.
- 👁️ **Fully Observable:** The agent has complete knowledge of the current state at all times.
- 🧘 **Static:** The world does not change except by the agent's actions.

In essence, **Classical Planning** is about **constructing a plan** — a **sequence of actions** — that achieves the desired goal **without needing to replan during execution**.

💡 Key Concepts

- **State Space Representation:** The problem is represented as a set of states and actions.
- **Transition Model:** Defines how each action transforms one state into another.
- **Goal Test:** Specifies the desired condition to be achieved.
- **Plan:** A sequence of actions that takes the agent from the initial state to the goal state.

Classical Planning Defined 🎯

Classical Planning is the task of finding a sequence of actions to accomplish a goal in a discrete, deterministic, static, fully observable environment.

⚙️ Challenges (Before PDDL)

🚫 Required **ad hoc heuristics** for each domain
🚫 Needed **explicit enumeration** of exponentially large state spaces
🚫 **Action schemas** had to be manually defined for each variation

🧠 PDDL (Planning Domain Definition Language) Solution

✅ Provides a **factored representation** (states described by variables and values)
✅ Uses **generalized action schemas** — one definition can handle many cases
✅ Eliminates the need for **domain-specific heuristics**



🧭 In Summary

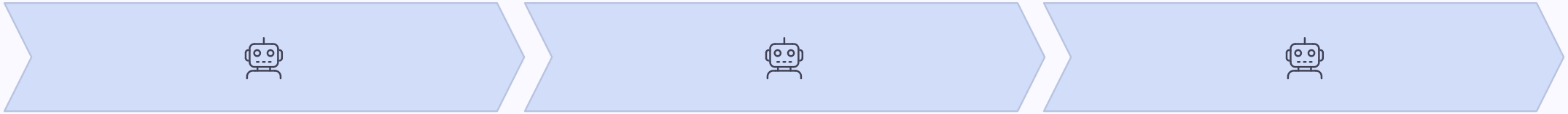
> **Classical Planning** 🧠 is the AI task of finding a **goal-achieving sequence of actions** in a **discrete, deterministic, fully observable, and static environment**, often formalized using **PDDL** to simplify and generalize the representation of planning problems.



STATE REPRESENTATION

State Transition Example

Watch how representation makes state changes explicit in a planning domain:



Initial State (S_0)

- At(Robot, Kitchen)
- Holding(Box)
- On(Book, Table)

The world's facts before an action.

Action: Move(Kitchen, Bedroom)

A defined action with preconditions and effects:

- Delete: At(Robot, Kitchen)
- Add: At(Robot, Bedroom)

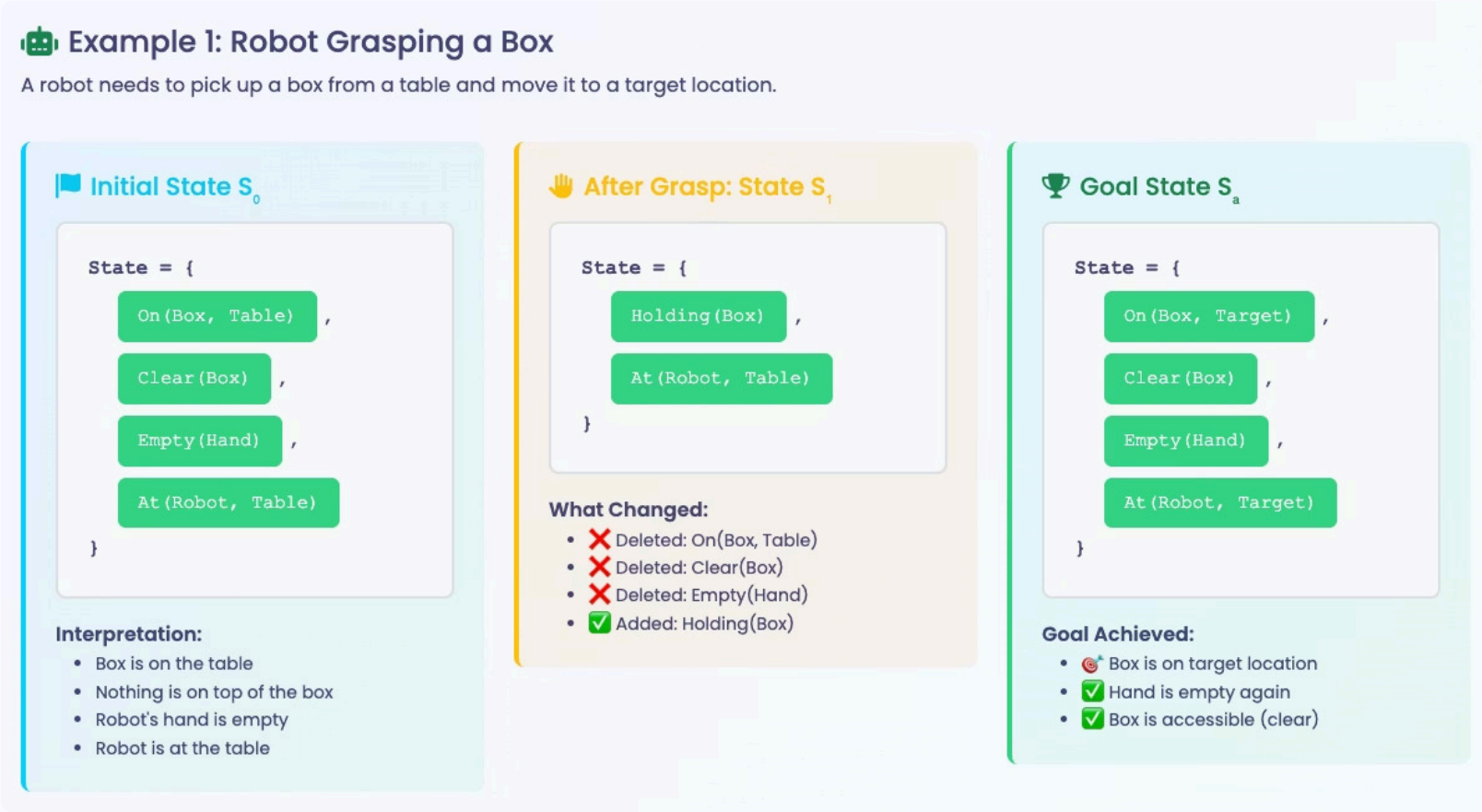
Resulting State (S_1)

- At(Robot, Bedroom) (new fact)
- Holding(Box)
- On(Book, Table)

The updated world facts after the action.

Notice how the representation makes it **crystal clear what changed** (robot's location) and **what stayed the same** (holding the box, book on table). This is the power of structured state representation!

Example Robot Grasping a Box



The Building Blocks of Planning States

What is a Ground Atomic Fluent?

Atomic

A single predicate (not a complex formula).

- ✅ `At(Robot, Kitchen)`
- ❌ `At(Robot, Kitchen) ∧ Holding(Box)`

Ground

Contains only constants (NO variables!).

- ✅ `On(Box, Table)`
- ❌ `On(x, Table)`

Fluent

Can change over time (unlike static facts).

- `At(Robot, Kitchen)` can become `At(Robot, Bedroom)` after an action.

Understanding these three properties is crucial for correctly modeling states in classical planning, ensuring that the system can precisely track and predict changes in the environment.

Valid Ground Atomic Fluents ✅

These examples illustrate correctly formed ground atomic fluents, crucial for representing discrete states in classical planning. Each fluent identifies a specific property or relationship involving concrete objects, ready for logical processing.

At(Robot, Kitchen)

Predicate: At

Constants: Robot, Kitchen

Holding(Box)

Predicate: Holding

Constants: Box

On(Book, Table)

Predicate: On

Constants: Book, Table

Clean(Room1)

Predicate: Clean

Constants: Room1

Empty(Hand)

Predicate: Empty

Constants: Hand

At(Plane, Riyadh)

Predicate: At

Constants: Plane, Riyadh

🚫 Invalid Fluents (❌ Not Allowed)

Invalid Form	Reason
`At(x, y)`	Contains variables — not *ground*
`¬Poor`	Negation not allowed in state description
`At(Spouse(Ali), Sydney)`	Uses a **function symbol** (not permitted in classical PDDL)

🧠 Quick Summary

> In PDDL, a **state** describes all true facts about the current world.
> The agent uses this **logical representation** to decide which actions to take next.

States as Database Tables 🗃️

As a Planning Set

```
State S = {
  At(Robot, Kitchen),
  Holding(Box),
  On(Book, Table),
  Clean(Kitchen)
}
```

A state is simply a collection of all currently true facts, where order doesn't matter and duplicates are implicitly handled.

As a Database Table

Predicate	Arguments
At	Robot, Kitchen
Holding	Box
On	Book, Table
Clean	Kitchen

This structure allows for clear representation and easy querying of current facts.

Key Properties

- States are **sets** (unordered, no duplicates)
- Adding/removing facts is like **INSERT/DELETE** in SQL
- Checking fact membership is like a **database query**

Closed-World Assumption (CWA)

> "What is not known to be true is FALSE"

In classical planning, we operate under the **Closed-World Assumption (CWA)**: any fact not explicitly listed in the current state is automatically considered **false**. There are no "unknowns" or "maybes" in our world model.

Important: Under CWA, we never use explicit negation like `¬At(Robot, Bedroom)`. The absence of a fact from the state's list is sufficient to declare it false.

Closed-World Assumption (CWA): Demonstrating CWA

```
State S = {
  At(Robot, Kitchen),
  Holding(Box)
}
```

Query	In State?	Truth Value (CWA)
`At(Robot, Kitchen)`	✓ YES	TRUE
`Holding(Box)`	✓ YES	TRUE
`At(Robot, Bedroom)`	NO	FALSE (by CWA)
`Clean(Kitchen)`	NO	FALSE (by CWA)
`Holding(Book)`	NO	FALSE (by CWA)

Important: Under CWA, we never use explicit negation like `¬At(Robot, Bedroom)`. The absence of a fact from the state's list is sufficient to declare it false.

CWA vs. Open-World Assumption

Closed-World Assumption (CWA)

In classical planning, what's not explicitly stated as true is false.

> Assumption: What's not stated is FALSE

State S = { At(Robot, Kitchen) }

Query: `At(Robot, Bedroom)`

Answer: FALSE

This simplifies planning by removing ambiguity; every fact has a definite truth value.

Open-World Assumption (OWA)

In general logic and database systems, what's not explicitly stated as true is unknown.

> Assumption: What's not stated is UNKNOWN

Knowledge Base KB = { At(Robot, Kitchen) }

Query: `At(Robot, Bedroom)`

Answer: UNKNOWN

Reflects incomplete information; absence of proof does not mean falsehood.

Unique Names Assumption (UNA)

> "Different names refer to different objects."

In classical planning, we adhere to the **Unique Names Assumption (UNA)**: each constant name refers to a unique object. If two names are different, they refer to different things – there are no aliases.

Valid under UNA ✅

- `At(Robot1, Kitchen)`
- `At(Robot2, Bedroom)`

`Robot1` and `Robot2` are distinct entities, so they can exist in different locations simultaneously without conflict.

Impossible under UNA ❌

- `At(Robot, Kitchen)`
- `At(Robot, Bedroom)`

Under UNA, a single `Robot` cannot logically be in two places at once. This state would be deemed inconsistent.

UNA: Real-World Example: Saudi Cities

Under UNA, the planning system understands:

- Riyadh ≠ Jeddah
- Riyadh ≠ Dammam
- Plane1 ≠ Plane2

Each name represents exactly one distinct geographical entity or aircraft, simplifying reasoning and preventing ambiguity.

What UNA Prevents:

- **Aliases:** Stops declarations like `Riyadh = CapitalCity`.
- **Same object, different names:** Ensures a strict one-to-one mapping between object names and entities.
- **Complex identity reasoning:** Reduces the need for sophisticated identity checks, unlike full First-Order Logic.

The goal is to keep the state representation simple and unambiguous: one unique name for one unique object.

Summary: Valid vs Invalid Fluents

Criterion	✓ VALID (Allowed)	✗ INVALID (Not Allowed)
Ground	`At(Robot, Kitchen)` Only constants	`At(x, Kitchen)` Contains variable 'x'
Atomic	`Holding(Box)` Single predicate	`Holding(Box) ∧ Clean(Table)` Complex formula with ∧
No Negation	`At(Robot, Room1)` Positive fact (use CWA for negation)	`¬At(Robot, Room2)` Explicit negation not allowed

No Quantifiers	`Clean(Room1)`
	`Clean(Room2)`
	List each fact explicitly
		`∀x Clean(x)`
		Universal quantifier not allowed

No Functions	`At(Robot, Location5)` Constants only	`At(Robot, NextRoom(Kitchen))` Function symbols not allowed



STRIPS (Stanford Research Institute Problem Solver)

🧩 STRIPS Representation and Syntax

🧠 What is STRIPS?

STRIPS (Stanford Research Institute Problem Solver) is a **formal model for classical planning** introduced in 1971.
It represents actions in terms of **preconditions**, **add lists**, and **delete lists** — defining how actions change the world.

⚙️ STRIPS Action Schema Structure

Component	Symbol / Example	Meaning
Action Name	Fly(p, from, to)	The action being performed
Preconditions	At(p, from) ∧ Plane(p)	Must be true before the action can execute
Add List	{At(p, to)}	Facts that become true after the action
Delete List	{At(p, from)}	Facts that become false after the action

💬 General STRIPS Syntax

At its core, STRIPS defines actions by detailing what conditions must be met for an action to occur, and how the world state changes afterward through literals being added or deleted.

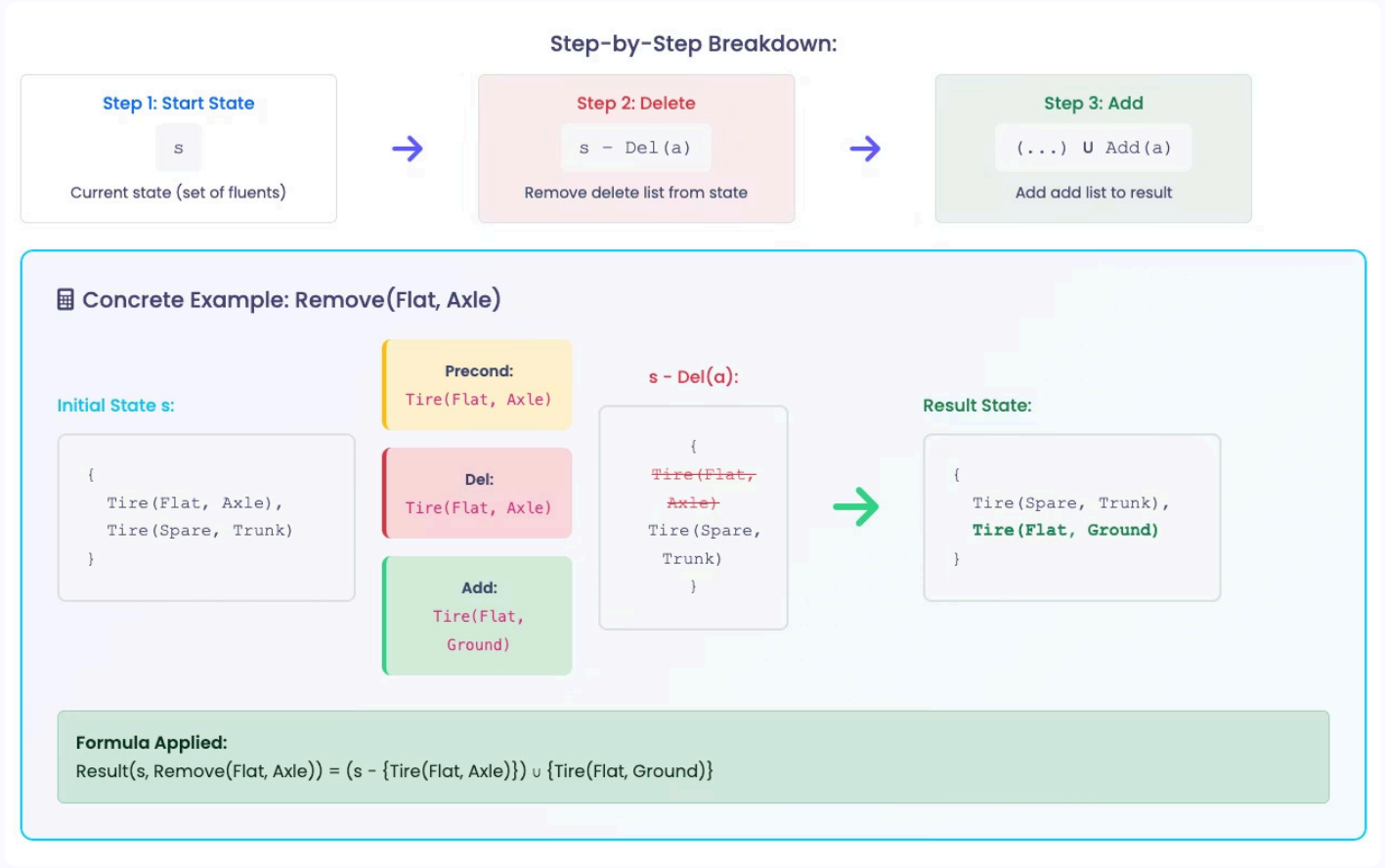
$$Action(A): \begin{cases} Precond(A) = \text{set of literals that must hold before A} \\ Add(A) = \text{literals added to the state} \\ Del(A) = \text{literals removed from the state} \end{cases}$$

🧮 State Transition Formula:

The execution of an action A in a state s transforms s into a new state by removing literals in $Del(A)$ and adding literals in $Add(A)$.

$$Result(s, A) = (s - Del(A)) \cup Add(A)$$

Example

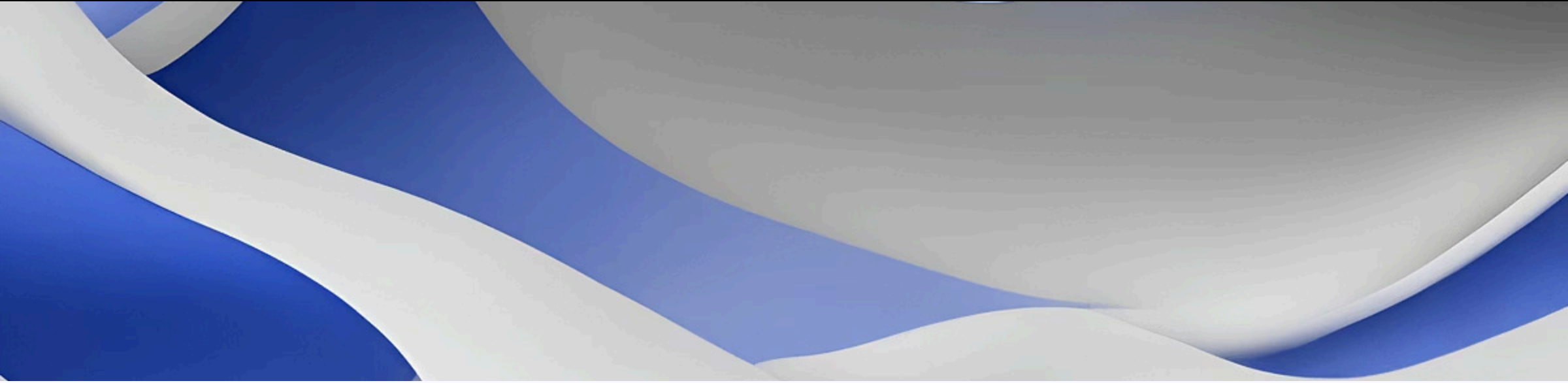

Step-by-Step Breakdown:
Step 1: Start State
s
Current state (set of fluents)
Step 2: Delete
s - Del(a)
Remove delete list from state
Step 3: Add
(...) U Add(a)
Add add list to result
Concrete Example: Remove(Flat, Axle)
Initial State s:
{
Tire(Flat, Axle),
Tire(Spare, Trunk)
}
Precond:
Tire(Flat, Axle)
Del:
Tire(Flat, Axle)
Add:
Tire(Flat, Ground)
s - Del(a):
{
~~Tire(Flat, Axle)~~
Tire(Spare, Trunk)
}
Result State:
{
Tire(Spare, Trunk),
Tire(Flat, Ground)
}
Formula Applied:
Result(s, Remove(Flat, Axle)) = (s - {Tire(Flat, Axle)}) ∪ {Tire(Flat, Ground)}

✈️ Example: Fly Action (STRIPS Form)

Component	Definition
Action:	`Fly(p, from, to)`
Precond:	`At(p, from)`
Add:	`At(p, to)`
Del:	`At(p, from)`

🔍 Key Points

- States are sets of **positive ground literals**.
- STRIPS assumes the **closed-world assumption** (what's not listed is false).
- Modern planners (via **PDDL**) extend STRIPS with **typing**, **quantifiers**, and **numeric fluents** — but still follow its **logical foundation**.



PDDL (Planning Domain Definition Language)

What is PDDL?

PDDL (Planning Domain Definition Language) is the standardized language used to represent planning problems in Artificial Intelligence. It significantly extends the foundational concepts of STRIPS with modern features like **types**, **variables**, and **structured syntax**.

> Think of PDDL as the "programming language" for planning – it provides a common, expressive format that any planning system can read and solve, fostering collaboration and research.

Historical Context & Evolution

1998

Created for the International Planning Competition (IPC)

Purpose

Standardize problem representation for planners

Foundation

Based on STRIPS (1971) + ADL extensions

Evolution

Current Version: PDDL 3.1 (supports preferences, constraints)

Why PDDL?

Before PDDL, every planner used its own unique input format. PDDL unified the field, enabling easy comparison and benchmarking of different planning algorithms.

🌍 State Representation in PDDL (Planning Domain Definition Language)

🧩 What is a State?

A **state** in PDDL is represented as a **conjunction of ground atomic fluents**:

$$State = F_1 \land F_2 \land \cdots \land F_n$$

Each fluent describes a fact that is **true** in the current world.

📖 Terminology

Term	Meaning	Example
Atomic	A single predicate (no logical connectives)	At(Truck1, Sydney)
Ground	Contains **no variables** — all arguments are constants	✅ At(Truck1, Sydney) ❌ At(x, y)
Fluent	A property that can **change over time**	Poor, At(Truck1, Sydney)

🧠 Connection Between PDDL and Formal Logic

💬 1. **Logical Foundations**

PDDL is based on **First-Order Predicate Logic (FOL)** — the same logic used in knowledge representation and reasoning. However, it uses a **simplified, specialized form** designed for planning tasks.

Concept	In Formal Logic	In PDDL
Predicates	Represent relationships between objects	`At(Truck1, Sydney)`
Constants	Refer to specific objects	`Truck1`, `Sydney`
Variables	Stand for any object	`x`, `y`
Logical Connectives	$\wedge$, $\vee$, $\neg$, $\rightarrow$	Used in preconditions and effects
Quantifiers	$\forall$, $\exists$	Rarely used — most formulas are **grounded**
Functions	Return values from objects	❌ *Not allowed* in classical PDDL

🧠 Connection Between PDDL and Formal Logic

⚙️ 2. How PDDL Simplifies Logic

To make **automated reasoning feasible**, PDDL **restricts** full first-order logic:

- ❌ **No functions** (to keep the world finite)
- ✅ **Only positive literals** in state descriptions (negations are implicit via the **Closed-World Assumption**)
- ✅ **Ground atoms only** — no variables in actual states
- ✅ **Conjunctions of facts** — no disjunctions or quantifiers in states

> ⚠️ 👉 These simplifications turn the planning problem into something computers can efficiently reason about (using search, not theorem proving).

🧩 3. Logical Semantics of States

Each **PDDL state** can be viewed as a **model** in formal logic:

- A **state** = a set of **ground atoms** that are true.
- The **closed-world assumption** means everything else is false.
- A **plan** is a sequence of actions that transforms one logical model (state) into another where the **goal formula** is satisfied.

🧱 4. Relation to Logic in AI

Logic Topic	Connection to PDDL
Propositional Logic	Each fluent acts like a propositional symbol (`At(Truck1, Sydney)` = True/False)
First-Order Logic	PDDL's syntax comes from FOL (predicates, arguments)
Inference	Planning uses *logical consequence* — actions cause predictable state transitions
Model Theory	Each state is a *model* that satisfies certain facts

✅ 🧭 **In Summary:**
PDDL is a **logical language** — a **practical subset of first-order logic** tailored for planning.
It keeps the **expressiveness** of FOL predicates but **simplifies** reasoning to enable **efficient plan search** rather than full logical inference.

🧠 1. What Is PDDL Syntax?

PDDL syntax is **Lisp-like**, meaning everything is enclosed in **parentheses (...)**
It's both **structured** and **human-readable**, designed for **automated planners** to parse easily.

Every PDDL file is either:

- A **domain file** → defines *what can happen* (actions, predicates, objects' types)
- A **problem file** → defines *what is true right now* and *what we want to achieve*

📁 2. PDDL File Structure Overview

🗂️ Domain File (`domain.pddl`)

Defines the **rules of the world** — *the verbs (actions)* the agent can use.

```
(define (domain flight-domain)
  (:requirements :strips :typing)
  (:types plane airport city)
  (:predicates
    (At ?p - plane ?a - airport)
    (Airport ?a - airport)
    (Plane ?p - plane)
  )
  (:action Fly
    :parameters (?p - plane ?from - airport ?to - airport)
    :precondition (and (At ?p ?from) (Plane ?p) (Airport ?from) (Airport ?to))
    :effect (and (not (At ?p ?from)) (At ?p ?to))
  )
)
```

✅ Explanation:

- `:types` — declares object types (optional)
- `:predicates` — defines what can be *true or false* (fluents)
- `:action` — defines action schemas (with preconditions & effects)
- Keywords like `:parameters`, `:precondition`, `:effect` are fixed syntax.

🧩 Problem File (problem.pddl)

🗂️ Domain File (domain.pddl)

Defines the **specific instance** — *the nouns (objects)* and *goals*.

```
(define (problem flight-problem)
  (:domain flight-domain)
  (:objects
    Plane1 - plane
    Riyadh Jeddah - airport
  )
  (:init
    (At Plane1 Riyadh)
    (Airport Riyadh)
    (Airport Jeddah)
    (Plane Plane1)
  )

  (:goal
    (At Plane1 Jeddah)
  )
)
```

✅ Explanation:

- `:domain` → links this problem to its domain definition.
- `:objects` → declares the specific constants used.
- `:init` → lists all fluents true in the *initial state*.
- `:goal` → logical formula that must be true when planning succeeds.

🔤 3. Common Syntax Elements

Element	Syntax	Example
Domain definition	(define (domain <name>))	(define (domain logistics))
Problem definition	(define (problem <name>))	(define (problem truck-delivery))
Types	(:types <object-types>)	(:types truck package location)
Predicates	(:predicates (<predicate-name> <parameters>))	(At ?t - truck ?l - location)
Action schema	(:action <name> :parameters (...) :precondition (...) :effect (...))	see Fly example
Initial state	(:init ...)	(At Truck1 Riyadh)
Goal condition	(:goal ...)	(At Truck1 Jeddah)

⚙️ 4. Logical Foundation

Under the hood, PDDL uses:

- **Conjunctions** → `(and ...)`
- **Negations** → `(not (...))`
- **Variables** → start with `?` (e.g., `?x`, `?truck`)
- **Constants** → lowercase names (e.g., `truck1`, `riyadh`)
- **Types** (optional) → declared with `-` (e.g., `?t - truck`)

🧭 5. Example: Saudi Delivery Domain

> 🧠 **Planner's task:**
> Find a sequence of `Drive` actions that moves `Truck1` from **Riyadh → Dammam**.

🗂️ *Domain:*

```
(define (domain saudi-delivery)
  (:requirements :strips :typing)
  (:types truck city)

  (:predicates
    (At ?t - truck ?c - city)
    (Connected ?from - city ?to - city)
  )

  (:action Drive
    :parameters (?t - truck ?from - city ?to - city)
    :precondition (and (At ?t ?from) (Connected ?from ?to))
    :effect (and (not (At ?t ?from)) (At ?t ?to))
  )
)
```

🧩 *Problem:*

```
(define (problem delivery-instance)
  (:domain saudi-delivery)

  (:objects
    Truck1 - truck
    Riyadh Jeddah Dammam - city
  )

  (:init
    (At Truck1 Riyadh)
    (Connected Riyadh Jeddah)
    (Connected Jeddah Dammam)
  )

  (:goal
    (At Truck1 Dammam)
  )
)
```

🎯 Summary

Concept	Description
Syntax Style	Lisp-based (parentheses, keyword structure)
Domain File	Defines types, predicates, and actions
Problem File	Defines objects, initial state, and goal
Planner Output	A sequence of ground actions (a plan)

🚚 PDDL Example: State Representation in a Logistics Domain

🎯 Problem Statement (Agent Context)

An **AI logistics agent** manages delivery trucks operating across major Saudi cities.
The agent's goal is to **plan routes and deliveries** efficiently by reasoning about the current **state of the world** — where trucks are located, what they are carrying, and their conditions.

🧩 Example PDDL States

Description	PDDL State Representation
Agent is poor and unknown (a simple symbolic example)	$Poor \wedge Unknown$
Two trucks are located in different cities	$At(Truck1, Riyadh) \wedge At(Truck2, Jeddah)$

💡 PDDL Example States

🧮 Database Semantics (PDDL Assumptions)

- ✅ **Closed-World Assumption (CWA):**
  If a fluent is **not listed**, it is **assumed false**.

> Example: If `At(Truck1, Dammam)` is not mentioned ⇒ the truck is *not in Dammam*.

- 🔠 **Unique Names Assumption (UNA):**
  Different names refer to **distinct objects**.

> Example: `Truck1 ≠ Truck2`, `Riyadh ≠ Jeddah`.

💡 Interpretation

- Each line in the state represents a **fact that is true** in the world.
- All other possible facts are **false by default** (CWA).
- The planner uses these facts to **reason about actions**, such as:

```
(Move Truck1 Riyadh Dammam)
(Load Package1 Truck1 Riyadh)
```

SUMMARY

State Representation in PDDL 📊

In PDDL, a **state** is represented as a conjunction of ground atomic fluents. "Ground" means no variables, "fluent" means an aspect of the world that changes over time, and "atomic" means a single predicate with constant arguments.

Example States

$Poor \wedge Unknown$ represents a hapless agent's state

$At(Truck_1, Melbourne) \wedge At(Truck_2, Sydney)$ represents truck locations

Database Semantics

Closed-world assumption: Unmentioned fluents are false

Unique names assumption: Different names mean different objects

Invalid Fluents ❌

$At(x, y)$ - contains variables

$\neg Poor$ - is a negation

$At(Spouse(Ali), Sydney)$ - uses function symbol

🛫 Action Schemas in PDDL

🧠 What is an Action Schema?

An **action schema** defines a *template* (or *family*) of possible actions.
Each schema describes how performing the action changes the world — once its **preconditions** are satisfied.

It's a **generalized rule** that can later be instantiated (grounded) with actual objects.

🧩 Structure of an Action Schema

An action schema has **four key components**:

Component	Description	Example
Action Name	Describes the action	Fly(p, from, to)
Variables	Parameters used in the action	`p`, `from`, `to`
Precondition	Must be **true** before the action can be applied	At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
Effect	Describes what **changes** after the action	¬At(p, from) ∧ At(p, to)

🛫 Action Schemas in PDDL Example

✈️ Example: Flying a Plane

```
Action(Fly(p, from, to),
  PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
  EFFECT: ¬At(p, from) ∧ At(p, to)
)
```

💬 **Interpretation:**

A plane `p` can **fly** from city `from` to city `to`
— only if it's **at the departure airport** and both locations are valid **airports**.

🧭 Summary

> An **Action Schema** is a logical rule describing how the world changes.
> It connects **states** through **actions** — the foundation of **classical planning** in PDDL.

🔄 State Transition Formula

$$Result(s, a) = (s - Del(a)) \cup Add(a)$$

💬 **Meaning:**

To compute the new state after performing action `a` in state `s`:

1. **Remove** all facts in the delete list (`Del(a)`)
2. **Add** all facts in the add list (`Add(a)`)

🧩 **Example:**

```
Current state s: {At(Plane1, Riyadh), Airport(Riyadh),
Airport(Dammam)}
Action: Fly(Plane1, Riyadh, Dammam)

Result(s, a):
= (s - {At(Plane1, Riyadh)}) ∪ {At(Plane1, Dammam)}
```

✅ New state: `{At(Plane1, Dammam), Airport(Riyadh), Airport(Dammam)}`

⚙️ Key Components Explained

Concept	Meaning	Example
🧱 **Precondition**	Facts that must hold before the action executes	At(Plane1, Riyadh)
🪄 **Effect**	Facts that change after the action executes	Remove At(Plane1, Riyadh), add At(Plane1, Dammam)
➕ **Add List**	Positive effects — facts **added** to the state	{At(p, to)}
➖ **Delete List**	Negative effects — facts **removed** from the state	{At(p, from)}

Action Schemas (SUMMARY)🔧

An **action schema** represents a family of ground actions. Each schema consists of the action name, variables, a precondition, and an effect.

```
Action(Fly(p, from, to),
  PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
  EFFECT: ¬At(p, from) ∧ At(p, to))
```

Key Components

- **Precondition:** Conditions that must be true for action to apply
- **Effect:** Changes to the state after action execution
- **Add list:** Positive literals added to state
- **Delete list:** Negative literals removed from state

State Transition Formula

$$Result(s, a) = (s - Del(a)) \cup Add(a)$$

The resulting state is formed by removing delete list items and adding add list items to the current state.



Example: Air Cargo Transport ✈️

✈️ Problem Description

A **cargo transport agent** is responsible for delivering packages between major Saudi airports using two planes.
The agent must plan the sequence of **Load**, **Unload**, and **Fly** actions to achieve the delivery goals efficiently.

🏁 Initial State

Cargo	Location	Plane	Location
C_1	Riyadh	P_1	Riyadh
C_2	Jeddah	P_2	Jeddah

🎯 Goal State

- C_1 at **Jeddah**
- C_2 at **Riyadh**

🧩 Available Actions

Action	Description
Load(c, p, a)	Load cargo **c** into plane **p** at airport **a**
Unload(c, p, a)	Unload cargo **c** from plane **p** at airport **a**
Fly(p, from, to)	Fly plane **p** from airport **from** to airport **to**



Example: Air Cargo Transport ✈️

✈️ Problem Description

A **cargo transport agent** is responsible for delivering packages between major Saudi airports using two planes.
The agent must plan the sequence of **Load**, **Unload**, and **Fly** actions to achieve the delivery goals efficiently.

🧩 Available Actions

Action	Description
Load(c, p, a)	Load cargo **c** into plane **p** at airport **a**
Unload(c, p, a)	Unload cargo **c** from plane **p** at airport **a**
Fly(p, from, to)	Fly plane **p** from airport **from** to airport **to**

Example: Air Cargo Transport ✈️

🚀 Solution Plan

1. **Load(C_1, P_1, Riyadh)**
2. **Fly(P_1, Riyadh, Jeddah)**
3. **Unload(C_1, P_1, Jeddah)**
4. **Load(C_2, P_2, Jeddah)**
5. **Fly(P_2, Jeddah, Riyadh)**
6. **Unload(C_2, P_2, Riyadh)**

🌍 Visualization (Conceptual)

Initial State:
C_1 → Riyadh P_1 → Riyadh
C_2 → Jeddah P_2 → Jeddah

Goal:
C_1 → Jeddah C_2 → Riyadh

💡 Key Insight

This problem demonstrates **sequential planning**:

- Actions change the **state** of the world.
- Each new state satisfies the **preconditions** of the next action.
- The final state satisfies the **goal conditions**.

Action Schemas: Air Cargo 📦

Action(Load(c, p, a),
  PRECOND: At(c, a) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
  EFFECT: ¬At(c, a) ∧ In(c, p))

Action(Unload(c, p, a),
  PRECOND: In(c, p) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
  EFFECT: At(c, a) ∧ ¬In(c, p))

Action(Fly(p, from, to),
  PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
  EFFECT: ¬At(p, from) ∧ At(p, to))

The At predicate means "available for use at a given location." Cargo ceases to be At anywhere when it is In a plane, and only becomes At the new airport when unloaded.



Problem Setup: Spare Tire Planning 🚗

🎯 Goal

Have a **good spare tire properly mounted** on the car's axle.
Formally:

$$Goal : On(Spare, Axle)$$

🚗 Initial State

Fact	Meaning
At(Flat, Axle)	The flat tire is currently mounted on the axle
At(Spare, Trunk)	The good spare tire is in the trunk
Tire(Flat), Tire(Spare)	Both objects are tires

Formally:

$$Initial : At(Flat, Axle) \land At(Spare, Trunk)$$

🔧 Available Actions (Action Schemas)

1

Remove Flat

```
Action(Remove(Flat, Axle),
  PRECOND: At(Flat, Axle) ∧ Tire(Flat)
  EFFECT: ¬At(Flat, Axle) ∧ At(Flat, Ground)
)
```

Removes the flat tire from the axle, placing it on the ground.

2

Remove Spare

```
Action(Remove(Spare, Trunk),
  PRECOND: At(Spare, Trunk) ∧ Tire(Spare)
  EFFECT: ¬At(Spare, Trunk) ∧ At(Spare, Ground)
)
```

Takes the good spare tire from the trunk, placing it on the ground.

3

Put On Spare

```
Action(PutOn(Spare, Axle),
  PRECOND: At(Spare, Ground) ∧ ¬At(Flat, Axle)
  EFFECT: ¬At(Spare, Ground) ∧ At(Spare, Axle)
)
```

Mounts the spare tire onto the axle, achieving the goal of having the spare tire correctly installed.



🧠 Solution Plan (Valid Sequence)

1️⃣ **Remove Flat** — take off the flat tire
2️⃣ **Remove Spare** — take the spare out of the trunk
3️⃣ **Put On Spare** — mount the spare on the axle

✅ **Goal Achieved:** `At(Spare, Axle)`

💬 Conceptual Summary

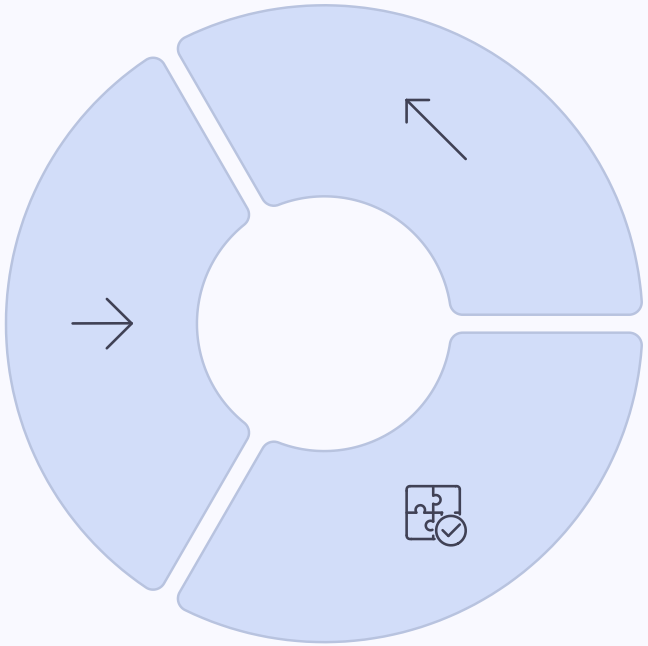
- Demonstrates **sequential planning**: actions must occur in order.
- Shows **state transitions** via logical preconditions and effects.
- Illustrates how **STRIPS-style operators** model real-world actions.

Planning Algorithms Overview 🔍

Planning = **searching for a sequence of actions** that leads from the **initial state** to the **goal state**.
Different algorithms approach this search in different directions and representations.

Forward Search

Progress from initial state through state space toward goal



Backward Search

Regress from goal through state descriptions to initial state

SAT-Based

Translate to Boolean satisfiability and use SAT solvers

Forward State-Space Search 🎯

Forward search applies heuristic search algorithms from earlier chapters, searching through ground states where every fluent is either true or false.

Key Concepts

- **Ground states:** All fluents are fully instantiated
- **Applicable actions:** Actions whose preconditions are satisfied
- **Unification:** Match current state against action preconditions

Challenge: Branching Factor

In a cargo problem with 10 airports, 50 planes, and 200 packages, there can be 450 to 10,450 possible actions per state!

Solution: Strong domain-independent heuristics make forward search feasible.

🔷 1. Forward Search (Progression Planning)

Core Idea:
Start from the **initial state** and apply **actions** step by step until the **goal** is reached.

$Initial \rightarrow actionsGoalInitial \xrightarrow{actions} Goal$InitialactionsGoal

How it works:

- Apply applicable actions (those whose preconditions hold).
- Generate successor states.
- Continue until a state satisfies the goal.

Example:
Like simulating actions in the real world — "What can I do next?"

✅ **Advantages:**

- Natural for **execution** and **simulation**.
- Works well with **heuristic search** (A*, FF planner).
- Easy to detect **dead ends**.

⚠️ **Limitations:**

- Large **branching factor** (many possible actions).
- Hard to focus search when **goals are far away**.

🔑 Key Concepts

Concept	Meaning
Ground States	Every fluent (fact) is fully instantiated — no variables. Example: `At(Plane1, Riyadh)` ✅
Applicable Actions	Actions whose **preconditions** are true in the current state.
Unification	Matching current state facts to an action's preconditions (logical pattern matching).

⚠️ Main Challenge: The Branching Factor

In realistic domains, the number of **possible actions per state** explodes! 💣

Example:

Cargo domain with

✈️ 10 airports,

🛩️ 50 planes,

📦 200 packages

→ **450 to 10,450 possible actions per state!**

That's an enormous search space to explore. 🌀

💡 Solution: Use Heuristics

To make forward search **feasible**, planners rely on **strong, domain-independent heuristics**, such as:

- **Relaxed planning heuristics** (ignore delete lists)
- **Additive and max heuristics**
- **Landmark heuristics**

These guide the search efficiently toward the goal without brute force.

🧭 Summary

> Forward search = exploring **ground states** using **heuristic guidance**.
> It's intuitive and general, but requires **powerful heuristics** to handle large state spaces.

Backward Search (Regression) 🔙

Regression search starts at the goal and applies actions backward until reaching the initial state. This considers only **relevant actions** rather than all applicable actions.

Relevant Action

An action with an effect that unifies with a goal literal, but with no effect that negates any part of the goal

Regression Formula

$$POS(g') = (POS(g) - ADD(a)) \cup POS(Precond(a))$$

$$NEG(g') = (NEG(g) - DEL(a)) \cup NEG(Precond(a))$$

Advantage

Keeps branching factor lower than forward search for most domains, focusing only on actions relevant to the goal

🔷 2. Backward Search (Regression Planning)

Core Idea:
Start from the **goal** and work *backward* — finding which actions could have produced the current goal state.

$$Goal \xleftarrow{regress} Initial$$

How it works:

- Choose a goal or subgoal condition.
- Find actions that can **achieve** part of that goal.
- Replace those goal conditions with their **preconditions**, forming new subgoals.
- Repeat until the **initial state** satisfies all subgoals.

✅ **Advantages:**

- Focuses directly on **goal-relevant actions**.
- Usually explores **fewer states** than forward search.
- Works well when goals are **specific** and **few**.

⚠️ **Limitations:**

- Regressing complex goals can be computationally expensive.
- Hard to represent large disjunctive preconditions.

Example:
Like reasoning in reverse: “What must have been true before this could happen?”

🎯 Relevant Action

An action **a** is *relevant* to a goal **g** if:

- ✅ One of its **effects** helps achieve a **goal literal** (matches/unifies with a subgoal).
- 🚫 None of its effects **contradict** any part of the goal.

Example:
If the goal is `At(C1, Riyadh)`, then `Unload(C1, P1, Riyadh)` is **relevant**,
but `Fly(P1, Riyadh, Jeddah)` is **not**.

🧮 Regression Formula

For a goal description **g** and action **a**, the *regressed goal* **g′** is computed as:

$$POS(g') = (POS(g) - ADD(a)) \cup POS(Precond(a))$$

$$NEG(g') = (NEG(g) - DEL(a)) \cup NEG(Precond(a))$$

Where:

- **POS(g)** = positive goal literals
- **NEG(g)** = negative goal literals
- **ADD(a)**, **DEL(a)** = action effects
- **Precond(a)** = action preconditions

> 💡 Meaning:
> Replace achieved goals with what must have been true before the action.

⚙️ Why Use Regression?

Feature	Backward (Regression) Search	Forward Search
Direction	From goal → initial	From initial → goal
Action Focus	Only **relevant actions**	All **applicable actions**
Branching Factor	🔽 Usually smaller	🔼 Can be large
Goal-Oriented	Yes 🎯	No (explores widely)

✅ Advantages

- Reduces **branching factor** by focusing on **goal-relevant actions**.
- Naturally **goal-directed**, avoiding irrelevant exploration.
- Works well when goals are **specific and few**.

🧭 Summary

> **Regression planning** reasons backward from the goal, finding what must have been true before each action.
> It's **efficient and focused**, making it powerful for domains with **many possible actions** but **few goal literals**.

Backward Search Example 📚

Consider the goal of **owning a book with ISBN 9780134610993**.

With a trillion possible 13-digit ISBNs and the action:

> Action(Buy(i), PRECOND: ISBN(i), EFFECT: Own(i))

Forward Search ❌

Would enumerate 10 billion ground Buy actions

Computationally infeasible

Backward Search ✅

Unify $Own(9780134610993)$ with $Own(i')$

Regress to $ISBN(9780134610993)$

Check initial state - done! Just one action considered.



🔍 Why Forward Search Fails

Forward search explores all applicable actions from the initial state, trying to reach the goal.

With **1 trillion possible 13-digit ISBNs**, this would generate **1 trillion ground** `Buy` **actions** (e.g., `Buy(0000000000001)`, `Buy(0000000000002)`, etc.).

This immense branching factor makes it **computationally infeasible** to explore every possible book to find the desired one.

🔄 How Backward Search Solves It Efficiently

1. Start from Goal: Focus on the specific goal: `Own(9780134610993)`.

2. Find Relevant Action: Look for an action whose effect matches this goal. The `Action(Buy(i), ..., EFFECT: Own(i))` directly unifies with `Own(9780134610993)`.

3. Regress Goal: The goal `Own(9780134610993)` $\Rightarrow \text{requires } ISBN(9780134610993)$ (from the precondition of the `Buy` action).

4. Check Initial State: If `ISBN(9780134610993)` is in the initial state, the plan is simply `Buy(9780134610993)`.

This process considers only **one specific action**, making it highly efficient for goal-driven problems.

⚙️ Comparison

Approach	Process	Actions Considered	Result
Forward Search	Try all `Buy(i)` for every ISBN	~1 trillion	❌ Infeasible
Backward Search	Match `Own(9780134610993)` → regress	1	✅ Efficient solution

🧭 Key Insight

> **Backward search** focuses only on **goal-relevant actions**,
> avoiding the explosion of irrelevant possibilities that forward search must explore.

Planning as Boolean Satisfiability 🔢

SAT-based planners like **SATPLAN** translate PDDL problems into propositional logic satisfiability problems for efficient SAT solvers.

01

Propositionalize Actions

Create ground propositions for each action schema with all constant substitutions

02

Add Exclusion Axioms

Ensure no two actions occur simultaneously

03

Add Precondition Axioms

If action taken at time t, preconditions must be true

04

Define Initial State

Assert all initial fluents and their negations

05

Add Successor-State Axioms

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$$

🔷 3. SAT-Based Planning

Core Idea:
Translate the **planning problem** into a **Boolean satisfiability (SAT)** formula,
then use a **SAT solver** to find a truth assignment that satisfies all constraints.

How it works:

- Encode each possible action and fluent as Boolean variables.
- Add logical constraints that capture preconditions, effects, and mutual exclusions.
- Use a SAT solver to check if a plan of *length k* exists.
- Incrementally increase k until a solution is found.

✅ **Advantages:**

- Leverages powerful **modern SAT solvers**.
- Guarantees **optimal (shortest) plans** if incrementally tested.
- Scales well for complex, highly constrained domains.

⚠️ **Limitations:**

- Requires **encoding step** (non-trivial).
- Doesn't produce intuitive intermediate reasoning (it's symbolic, not procedural).
- Works best for **fixed-horizon** planning.

🧩 Quick Comparison Table

Approach	Direction	Key Strength	Main Limitation	Typical Use
Forward Search	From initial → goal	Natural, easy to simulate	High branching factor	Online or heuristic planners
Backward Search	From goal → initial	Goal-directed	Complex regression	Goal-focused domains
SAT-Based	Symbolic (logical)	Exploits SAT solvers' power	Hard encoding	Large, structured problems

🎯 Most Important Takeaways

✅ **Forward Search** — Natural, heuristic-friendly (used in modern planners like FF, FastDownward).

✅ **Backward Search** — Efficient for few goals; focuses on relevance.

✅ **SAT-Based Planning** — Converts planning into logical satisfiability; powerful for optimization.

🧩 Planning as Boolean Satisfiability (SAT-Based Planning)

🧠 Core Idea

SAT-based planners (like 🧮 **SATPLAN**) transform a **PDDL planning problem** into a **propositional logic formula**. The planner then uses a **SAT solver** to find a truth assignment that represents a **valid plan**. If the SAT formula is **satisfiable**, the satisfying assignment corresponds to a **valid sequence of actions**.

⚙️ Key Steps in the Translation Process

Step	Description
1️⃣ **Propositionalize Actions**	Generate **ground propositions** for each action schema and all possible constant combinations. Example: `Fly(P1, Riyadh, Jeddah)`, `Load(C1, P1, Riyadh)`
2️⃣ **Add Exclusion Axioms**	Add constraints ensuring **mutual exclusivity** — no two conflicting actions occur simultaneously.
3️⃣ **Add Precondition Axioms**	If an **action** occurs at time *t*, its **preconditions** must be **true** at *t*.
4️⃣ **Define Initial State**	Encode all **initial fluents** (facts) as **true** and everything else as **false**.
5️⃣ **Add Successor-State Axioms (SSA)**	Define how fluents evolve from time *t* to *t+1*.

🔁 Successor-State Axioms (SSA)

A core component of SAT-based planning is defining how the state of the world evolves over time using Successor-State Axioms.

$$F^{t+1} \Leftrightarrow \text{ActionCausesF}^t \vee (F^t \wedge \neg \text{ActionCausesNotF}^t)$$

Meaning: A fluent **F** is true at time *t+1* **if and only if**:

- Some action occurring at time *t* **makes F true**.
- **F was already true** at time *t* and **no action** at time *t* makes it false.

🧮 Solving Process

1. Encode the planning problem as a **CNF formula** (for a fixed horizon k).
2. Run a **SAT solver** (e.g., MiniSAT, Glucose).
3. If satisfiable → extract plan of length k.
4. If not → increment k and repeat.

⚖️ Advantages of SAT-Based Planning

✅ Leverages **powerful modern SAT solvers**
✅ Produces **provably correct** and often **optimal** plans
✅ Handles **large, complex constraint interactions**

⚠️ **Limitations:**

- Requires **explicit grounding** of all actions
- Needs **fixed time horizon (k)**
- **Encoding** can be computationally expensive

🧭 Summary

> **SAT-based planning** = Encode planning as a **logical satisfiability problem**
> 🧠 Efficient, sound, and complete — driven by the power of **Boolean reasoning**.

END OF LECTURE



⚠ Additional Material - Outside Course Content

Other Classical Planning Approaches 🛠️

Other Classical Planning Approaches 🛠️

Planning Graph (Graphplan)

Uses specialized data structure to encode constraints on action relationships and mutual exclusions

Situation Calculus

Describes planning in first-order logic with successor-state axioms, contributing to theoretical understanding

CSP Encoding

Bounded planning as constraint satisfaction with single action variable per time step

Partial-Order Planning

Represents plans as graphs rather than linear sequences, useful for operations scheduling

Heuristics for Planning 

Factored representations enable **domain-independent heuristics** by defining relaxed problems that are easier to solve. An admissible heuristic never overestimates the cost to reach the goal.



Two Main Approaches

Add more edges to the graph (making paths easier to find) or group nodes together (creating abstractions with fewer states)



Ignore Preconditions

Drop all preconditions from actions, making every action applicable in every state



Ignore Delete Lists

Remove negative effects, enabling monotonic progress toward the goal

Ignore-Delete-Lists Heuristic 📈

The **ignore-delete-lists heuristic** removes all negative literals from action effects, creating a relaxed problem with no local minima.

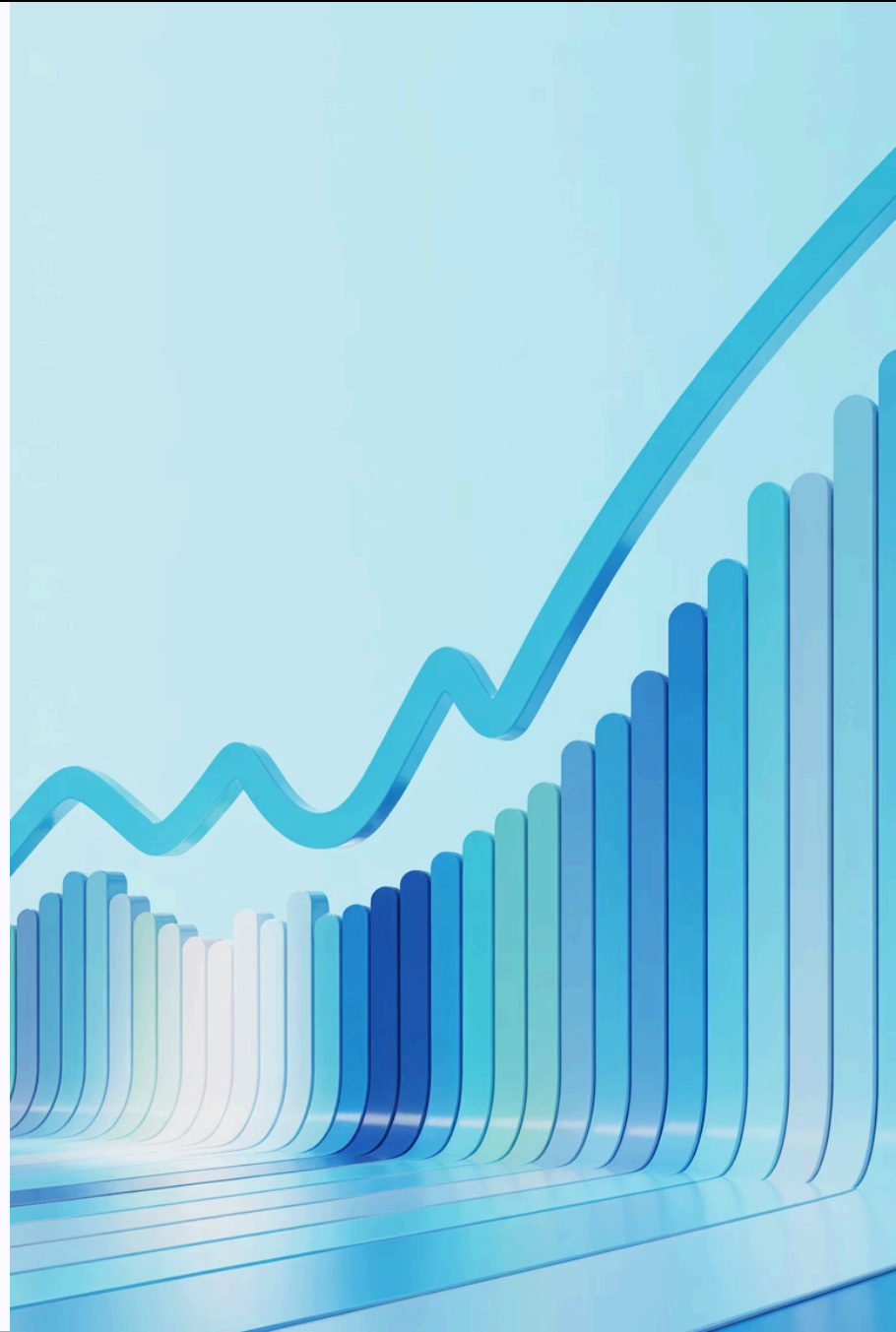
How It Works

- Remove delete lists from all actions
- Enables monotonic progress - no action undoes another
- Still NP-hard to find optimal solution
- Approximate solution found in polynomial time

Result

Creates wide paths to the goal with no dead ends. Simple hill-climbing search easily finds solutions without backtracking.

Used successfully in the FF (Fast Forward) planner.



Domain-Independent Pruning ✂️

Symmetry Reduction

Prune symmetric branches of search tree. Example: In blocks world with 12 blocks, placing any block x on any block y creates 110 symmetric states - only consider one.

Preferred Actions

Solve relaxed problem to get relaxed plan. Preferred actions are either steps in the relaxed plan or achieve preconditions of the relaxed plan.

Serializable Subgoals

When subgoals can be achieved in order without undoing previous subgoals. Example: Building a tower bottom-to-top in blocks world.

The Remote Agent planner for NASA's Deep Space One spacecraft exploited serializable subgoals to eliminate most search, enabling real-time spacecraft control.

State Abstraction in Planning 🗺️

State abstraction creates a many-to-one mapping from ground states to abstract states, reducing the number of states to search.

Example: Air Cargo

Original: 10 airports, 50 planes, 200 packages

States: $10^{50} \times (50 + 10)^{200} \approx 10^{405}$

Abstraction: Focus on 5 airports with packages

Reduced states: $10^{5} \times (5 + 10)^{5} \approx 10^{11}$

Key Concepts

Decomposition: Divide problem into parts, solve independently, combine

Subgoal independence: Cost of conjunction ≈ sum of individual costs

Pattern databases: Precompute and reuse across problems

Hierarchical Planning Introduction 🏗️

Complex problems require planning at multiple levels of abstraction. A high-level plan like "Go to San Francisco airport" decomposes into lower-level actions like "Order car, ride to airport."



Motor Control

Billions of primitive actions: "bend knee 5 degrees"



Physical Actions

Thousands of actions: "walk to car"



Transportation

Dozens of actions: "drive to airport"



Travel Plan

Few high-level actions: "go to airport, fly to destination"

High-Level Actions (HLAs) 🎯

A **high-level action (HLA)** has one or more **refinements** into sequences of actions. An **implementation** is a refinement containing only primitive actions.

```
Refinement(Go(Home, SFO),
  STEPS: [Drive(Home, SFOLongTermParking),
          Shuttle(SFOLongTermParking, SFO)])

Refinement(Go(Home, SFO),
  STEPS: [Taxi(Home, SFO)])
```

A high-level plan achieves the goal if **at least one** of its implementations achieves the goal. The agent chooses which implementation to execute.

HTN Planning Algorithm 🔄

Hierarchical Task Network (HTN) planning searches through refinements of high-level actions until finding a primitive solution.

1

Start with Act

Begin with top-level action

2

Choose HLA

Select high-level action to refine

3

Apply Refinement

Replace with lower-level sequence

4

Check Goal

If primitive and achieves goal, done!

Complexity Analysis

Non-hierarchical: $O(b^d)$

Hierarchical: $r^{(d-1)/(k-1)}$

Where r = refinements per HLA, k = actions per refinement

Key Insight

Small r and large k yield huge savings - taking the kth root of non-hierarchical cost!

Angelic Semantics 👼

Rather than exploring all implementations, **angelic semantics** allows reasoning about HLAs at an abstract level to prove plans work without considering implementations.

Demonic Nondeterminism 😈

Adversary chooses implementation - must work for ALL outcomes

Angelic Nondeterminism 👼

Agent chooses implementation - must work for AT LEAST ONE outcome

The **reachable set** REACH(s, h) is the set of states reachable by any implementation of HLA h from state s. A plan achieves the goal if its reachable set intersects the goal states.

HLA Effects with Angelic Semantics ✨

An HLA can have nine different effects on a fluent, using the˜symbol to mean "possibly, if the agent chooses."



$\tilde{+}A$

Possibly add A (leave unchanged or make true)



$\tilde{-}A$

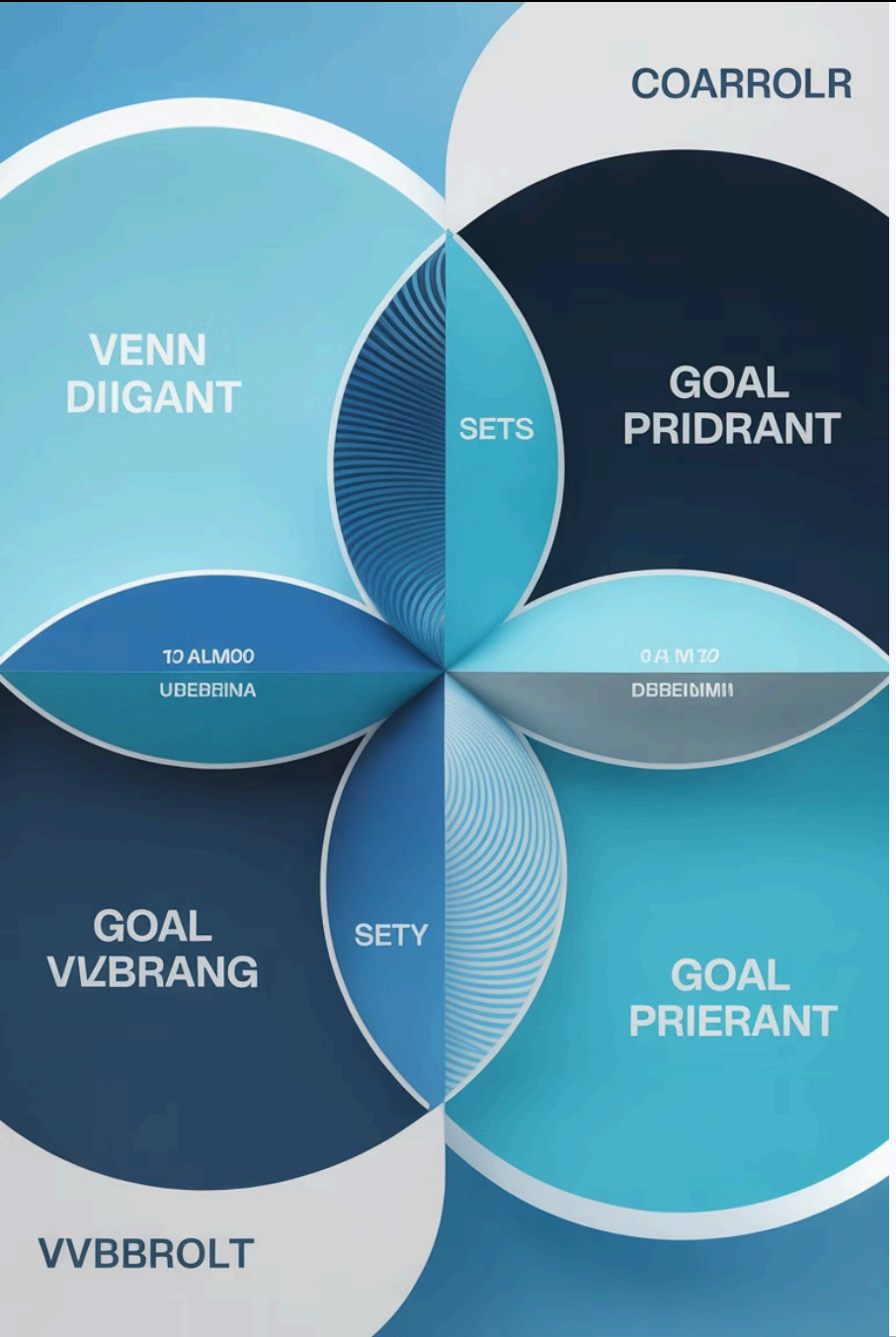
Possibly delete A (leave unchanged or make false)



$\tilde{\pm}A$

Possibly add or delete A (full control over value)

Example: $Go(Home, SFO)$ has effect $\tilde{-}Cash$ because the agent might choose to take a taxi (which costs cash) or drive (which doesn't).



Approximate Descriptions 📊

The Challenge

HLAs may have infinitely many implementations producing wiggly reachable sets. We need approximations:

- **Optimistic:** $REACH^+(s, h)$ may overstate reachable set
- **Pessimistic:** $REACH^-(s, h)$ may understate reachable set

$$REACH^-(s, h) \subseteq REACH(s, h) \subseteq REACH^+(s, h)$$

Testing Plans

- If optimistic set doesn't intersect goal → plan doesn't work ❌
- If pessimistic set intersects goal → plan works ✅
- If only optimistic intersects → uncertain, refine further 🤔

Angelic Search Algorithm 🔍

The **ANGELIC-SEARCH** algorithm commits to high-level plans that provably work while avoiding plans that don't, without considering implementations.

01

Check Optimistic Reachable Set

If doesn't intersect goal, plan fails - try another

02

Check Pessimistic Reachable Set

If intersects goal, plan works - commit to it!

03

Decompose Problem

Break into subproblems, one per plan step

04

Recursively Solve

Apply angelic search to each subproblem

This approach can provide exponential speedup by committing to abstract plans early and pruning away alternatives.

Nondeterministic Domains 🎲

Real-world environments are often partially observable, nondeterministic, or unknown. We extend planning to handle these challenges through three approaches:

Sensorless Planning

No observations available - must work for all possible states

Contingency Planning

Partial observability - create conditional plans with branches

Online Replanning

Unknown environment - monitor execution and replan as needed



Example: The Painting Problem 🎨

Goal: Make a chair and table match in color. Initial state: Two cans of paint, but colors of paint and furniture are unknown. Only the table is initially visible.

Actions Available

```
Action(RemoveLid(can),
  PRECOND: Can(can)
  EFFECT: Open(can))

Action(Paint(x, can),
  PRECOND: Object(x) ∧ Can(can) ∧
          Color(can, c) ∧ Open(can)
  EFFECT: Color(x, c))
```

Percept Schemas

```
Percept(Color(x, c),
 PRECOND: Object(x) ∧ InView(x))

Percept(Color(can, c),
 PRECOND: Can(can) ∧ InView(can)
 ∧ Open(can))
```



Sensorless Planning Solution 🔧

A **sensorless agent** has no percept schemas and must find a plan that works regardless of the actual colors.

1. **Open Can 1**
   RemoveLid(Can_1)
2. **Paint Chair**
   Paint(Chair, Can_1)
3. **Paint Table**
   Paint(Table, Can_1)

This solution coerces both pieces of furniture to be the same color (whatever color is in Can_1), even though the agent doesn't know what that color is! 🎨

Belief State Representation 🧠

In sensorless planning, we track **belief states** - sets of possible physical states the agent might be in.

Initial Belief State

$b_0 = Color(x, C(x))$

Objects and cans have colors (Skolemized)

After RemoveLid(Can_1)

$b_1 = Color(x, C(x)) \land Open(Can_1)$

After Paint(Chair, Can_1)

$b_2 = Color(x, C(x)) \land Open(Can_1) \land$
$Color(Chair, C(Can_1))$

After Paint(Table, Can_1)

$b_3 = Color(x, C(x)) \land Open(Can_1) \land$
$Color(Chair, C(Can_1)) \land Color(Table, C(Can_1))$

✅ Goal satisfied!

Belief State Update Rule 🔄

The family of belief states defined as **conjunctions of literals** is closed under updates by PDDL action schemas. This means any belief state can be represented compactly in $O(n)$ space for n fluents, rather than 2^n!

Update formula for belief state b after action a:

$$b' = RESULT(b, a) = (b - DEL(a)) \cup ADD(a)$$

Add List

If action adds ℓ, then ℓ will be true in b' regardless of initial value

Delete List

If action deletes ℓ, then ℓ will be false in b' regardless of initial value

Unaffected

If action doesn't affect ℓ, it retains unknown value and doesn't appear in b'

Conditional Effects Challenge ⚠️

The 1-CNF property breaks down when actions have **conditional effects** - effects that depend on the current state.

Vacuum World Example

```
Action(Suck,
  EFFECT: when AtL: CleanL
          when AtR: CleanR)
```

Applied to initial belief state True, results in:

$(AtL \wedge CleanL) \vee (AtR \wedge CleanR)$

No longer in 1-CNF! 😱



⚠ Conditional effects can induce arbitrary dependencies among fluents, leading to belief states of exponential size in the worst case.

Contingent Planning 🌳

Contingent planning generates plans with conditional branching based on percepts, appropriate for partially observable or nondeterministic environments.

[LookAt(Table), LookAt(Chair),
 if Color(Table,c) $\wedge$ Color(Chair,c) then NoOp
 else [RemoveLid(Can_1), LookAt(Can_1), RemoveLid(Can_2), LookAt(Can_2),
  if Color(Table,c) $\wedge$ Color(can,c) then Paint(Chair,can)
  else if Color(Chair,c) $\wedge$ Color(can,c) then Paint(Table,can)
  else [Paint(Chair,Can_1), Paint(Table,Can_1)]]]

Variables are existentially quantified. The agent maintains belief state as a logical formula and evaluates branch conditions by checking entailment.

Belief State Update with Percepts 👁️

Updating belief state after action and percept happens in two stages:

Stage 1: Action Effect

$\hat{b} = (b - DEL(a)) \cup ADD(a)$

Same as sensorless case

Stage 2: Incorporate Percept

Add percept literals $p_1, \ldots, p_k$ to belief state

Also add preconditions of percept schemas (since they must hold for percept to occur)

If multiple percept schemas could apply, must add disjunction of preconditions, which takes belief state outside 1-CNF.

Online Planning & Replanning 🔄

Online planning interleaves planning and execution, using execution monitoring to detect when replanning is needed.

Action Monitoring

Verify preconditions still hold before executing each action



Plan Monitoring

Verify remaining plan will still succeed before each action

Goal Monitoring

Check if there's a better set of goals to pursue

When Replanning is Needed 🚨

Incomplete Plans

Some branches simply say "Replan" - defer detailed planning until execution reaches that point

Missing Preconditions

Model may not know removing paint lid requires screwdriver

Missing Effects

Painting object may also get paint on floor

Missing Fluents

Model may lack notion of paint amount in can

Exogenous Events

Someone knocks over paint can, or goals change

⚠ Without monitoring and replanning ability, agent behavior is fragile and depends on absolute correctness of its model.

Plan Repair Strategy 🔧

When execution monitoring detects a problem, the agent finds a **repair** - a path from the observed state O back to some point P on the original plan.

Repair Process

1. Execute plan steps until expected state E
2. Observe actual state O (may differ from E)
3. Find point P on original plan to rejoin
4. Compute repair from O to P
5. Continue with plan from P to goal G

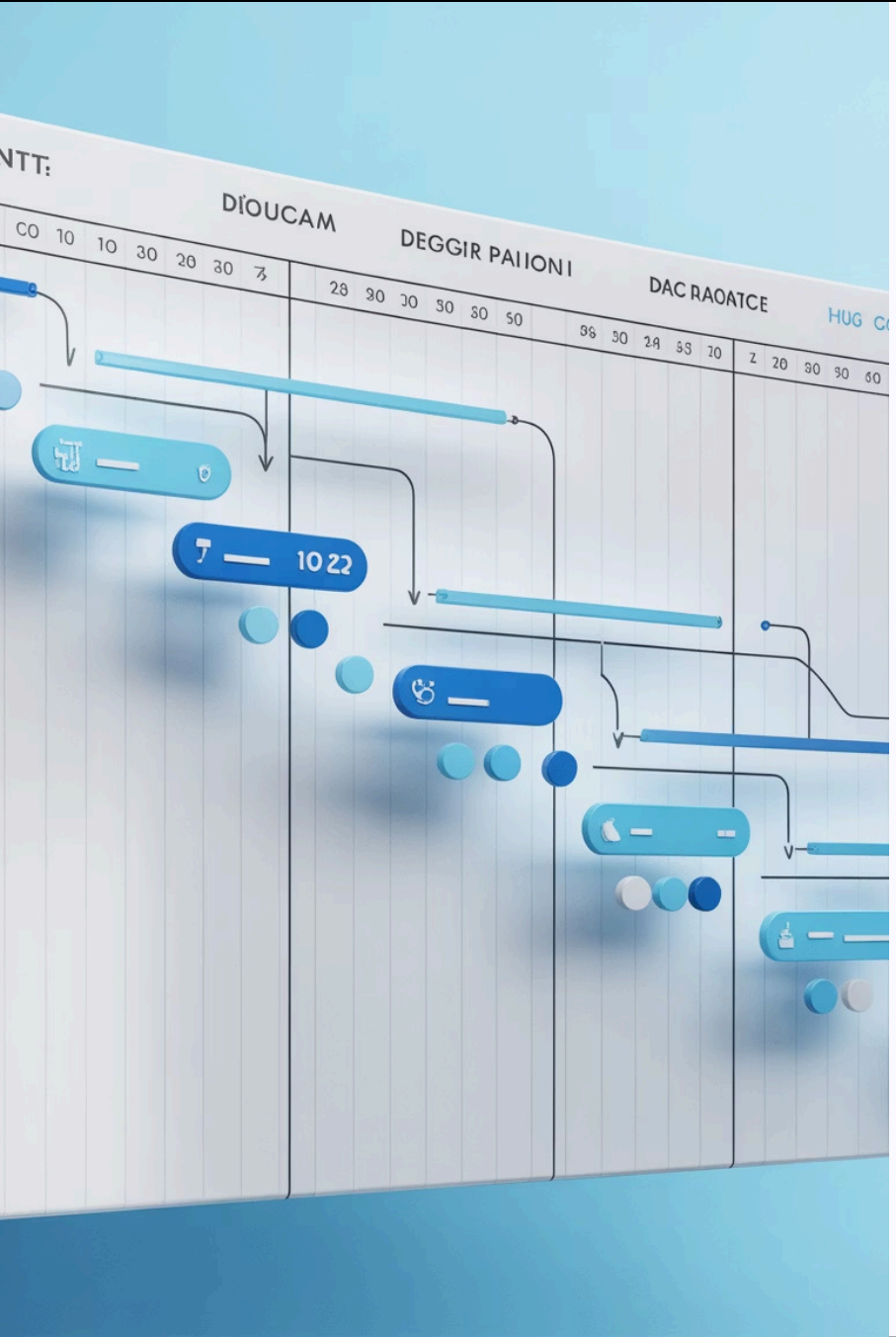
Cost Minimization

Minimize total cost:

Repair cost (O to P) + **Continuation cost** (P to G)

P might be the goal state itself if that's most efficient.





Time, Schedules, and Resources



Classical planning doesn't specify *when* actions occur or *how long* they take. **Scheduling** adds temporal information and handles resource constraints.

1

Planning Phase

Select actions with ordering constraints to meet goals

2

Scheduling Phase

Add temporal information and ensure resource constraints are satisfied

Job-Shop Scheduling Problem 🏭

A **job-shop scheduling problem** consists of jobs with actions, ordering constraints, durations, and resource requirements.

```
Jobs({AddEngine₁  AddWheels₁  Inspect₁},
   {AddEngine₂  AddWheels₂  Inspect₂})
Resources(EngineHoists(1), WheelStations(1),
       Inspectors(2), LugNuts(500))

Action(AddEngine₁, DURATION:30, USE:EngineHoists(1))
Action(AddWheels₁, DURATION:30,
     CONSUME:LugNuts(20), USE:WheelStations(1))
```

2	4	115
Jobs	Resource Types	Minutes
Assembling two cars	Hoists, stations, inspectors, nuts	Optimal makespan with constraints

Resource Types 📦

Consumable Resources

Used up by actions and not available afterward

Example: Lug nuts, fuel, raw materials

Reusable Resources

Occupied during action but available again after

Example: Engine hoists, pilots, tools

Aggregation

Group indistinguishable objects into quantities (e.g., Inspectors(2)) rather than named entities

Essential for reducing complexity!

Critical Path Method 🛤️

The **critical path method (CPM)** determines earliest and latest start times for actions to minimize makespan.

Key Concepts

- **Critical path:** Longest duration path through plan
- **Slack:** LS - ES, the flexibility in scheduling
- **ES (Earliest Start):** Earliest possible start time
- **LS (Latest Start):** Latest start without delaying plan

Formulas

$$ES(Start) = 0$$

$$ES(B) = \max_{A \prec B} ES(A) + Duration(A)$$

$$LS(Finish) = ES(Finish)$$

$$LS(A) = \min_{B \succ A} LS(B) - Duration(A)$$

Complexity: $O(Nb)$ where N = number of actions, b = max branching factor

Scheduling with Resource Constraints 📊

Adding resource constraints makes scheduling NP-hard because "cannot overlap" constraints are disjunctions of linear inequalities.



Without Constraints

Minimum duration achievable

With Constraints

30 minutes longer due to resource conflicts

Minimum slack heuristic: Schedule action with least slack first. Greedy approach that often works well but may not find optimal solution.

Planning Approaches Analysis 📈

Planning combines search and logic to control combinatorial explosion. With n propositions, there are 2^n states!

Key Success Factors

Identification of independent subproblems provides exponential speedup. Factored representations enable domain-independent heuristics.

Current Landscape

No clear winner - different techniques work best on different problems. Portfolio systems run multiple algorithms in parallel or selectively.

Future Directions

Synthesis of expressive first-order/hierarchical representations with efficient factored/propositional representations.

Key Takeaways 🎓

Factored Representations

PDDL enables domain-independent heuristics and powerful algorithms by explicitly representing states and actions

Multiple Approaches

Forward/backward search, SAT encoding, CSP, HTN planning - each has strengths for different problem types

Hierarchical Planning

Angelic semantics allows provably correct high-level plans without considering implementations

Handling Uncertainty

Sensorless, contingent, and online planning address partial observability and nondeterminism

Real-World Applications

Scheduling with time and resource constraints enables practical industrial applications

Ongoing Research

Integration with learning, stochastic environments, and multi-agent systems continues to advance the field